EDUPATH STUDENT CAREER PORTAL · SERVICE COMPLETION RECORD

FROZEN

student-profile-svc

EduPath :: Student Profile Service

13/13 Tests Passing · Hexagonal · Kafka · PostgreSQL

PLATFORM	STACK	ARCHITECTURE	GENERATED
EduPath v1.0	Java 17 · Spring Boot 3.3.5	Hexagonal · Kafka · PostgreSQL	March 07, 2026

BUILD STATUS: SUCCESS · ALL TESTS PASSING · RECORD FROZEN

student-profile-svc — COMPLETION RECORD

STATUS: FROZEN — DO NOT MODIFY
Tests passed: 13/13 | Build: SUCCESS | Date: 2026-03-07

Service Overview

EduPath Student Career Portal — foundational student identity and academic history service. Manages student profiles, academic records with subject scores, and target exam selections. Publishes domain events to Kafka on every state change so downstream services (performance-svc, ai-mentor-svc) can react.

Test Results

```
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS — Total time: 11.603 s
```

TEST CLASS	TESTS	RESULT
ArchitectureRulesTest	5	PASS
StudentProfileServiceTest	6	PASS
TargetExamServiceTest	2	PASS

Architecture

Standard hexagonal (ports & adapters). `domain` has zero Spring dependencies.

```
com.edutech.student/
  StudentProfileSvcApplication.java
  api/
    StudentProfileController.java      REST — /api/v1/students/**
    AcademicRecordController.java      REST — /api/v1/students/{id}/academic-records/**
    TargetExamController.java          REST — /api/v1/students/{id}/target-exams/**
    GlobalExceptionHandler.java        RFC 7807 ProblemDetail
  application/
    service/
      StudentProfileService.java       CreateStudentProfileUseCase, GetStudentProfileUseCase, Updat
eStudentProfileUseCase
      AcademicRecordService.java       AddAcademicRecordUseCase
      TargetExamService.java           SetTargetExamUseCase
      DashboardService.java            GetStudentDashboardUseCase
    dto/                               Java records (immutable)
    config/JwtProperties.java          record, @ConfigurationProperties("jwt")
    exception/                         StudentNotFoundException, DuplicateStudentException, etc.
  domain/
    model/
      StudentProfile.java              @Entity, static factory, @Version, soft delete
      AcademicRecord.java              @Entity, @OneToMany SubjectScore
      SubjectScore.java                @Entity, @Embeddable grade detail
      TargetExam.java                  @Entity, partial unique index (student_id, exam_code, delete
d_at IS NULL)
    model/enums/                       Board, Stream, ExamCode, Gender, ProfileStatus
    event/                             StudentProfileCreatedEvent, AcademicRecordAddedEvent, Target
ExamSetEvent, StreamSelectedEvent
    port/in/                           6 use-case interfaces
    port/out/                          StudentProfileRepository, AcademicRecordRepository, TargetEx
amRepository, StudentEventPublisher
  infrastructure/
    persistence/                       SpringData*Repository (package-private) + public *Persistenc
eAdapter @Component
    kafka/StudentEventKafkaAdapter.java  KafkaTemplate publisher, best-effort (failures logged)
    security/
```

```
      SecurityConfig.java              Trusts X-User-Id / X-User-Role from student-gateway (no JWT
re-validation)
      JwtAuthenticationFilter.java     Reads X-User-Id header from gateway
    config/
      ApplicationConfig.java
      KafkaTopicProperties.java        @ConfigurationProperties("kafka.topics")
```

Flyway Migrations

VERSION	SCRIPT	DESCRIPTION
V1	V1__create_student_schema.sql	CREATE SCHEMA student_schema
V2	V2__create_students.sql	students table
V3	V3__create_academic_records.sql	academic_records + subject_scores
V4	V4__create_target_exams.sql	target_exams + partial unique index

API Endpoints

METHOD	PATH	USE CASE
POST	`/api/v1/students`	Create student profile
GET	`/api/v1/students/{id}`	Get profile by ID
PATCH	`/api/v1/students/{id}`	Update profile
GET	`/api/v1/students/{id}/dashboard`	Student dashboard
POST	`/api/v1/students/{id}/academic-records`	Add academic record
GET	`/api/v1/students/{id}/academic-records`	List academic records
POST	`/api/v1/students/{id}/target-exams`	Set target exam
GET	`/api/v1/students/{id}/target-exams`	List target exams

Kafka Events Published

EVENT	TOPIC	TRIGGER
StudentProfileCreatedEvent	`student.profile.created`	Profile creation

EVENT	TOPIC	TRIGGER
AcademicRecordAddedEvent	`student.academic.record.added`	Academic record added
TargetExamSetEvent	`student.target.exam.set`	Target exam set
StreamSelectedEvent	`student.stream.selected`	Stream selected on profile

Key Design Decisions

- **Security**: Trusts `X-User-Id` / `X-User-Role` headers from student-gateway. No JWT re-validation downstream.
- **Duplicate detection**: `DuplicateStudentException` if student with same `userId` already exists (unique constraint on `user_id`).
- **Target exam uniqueness**: Partial unique index `(student_id, exam_code)` WHERE `deleted_at IS NULL` — allows re-enrollment after soft delete.
- **No Lombok**: Manual constructors throughout. Constructor injection only.
- **Optimistic locking**: `@Version` on `StudentProfile`, `AcademicRecord`, `TargetExam`.
- **Kafka best-effort**: Publisher failures logged, never roll back DB transaction.

ArchUnit Rules (5)

1. `domain` package has no dependencies on Spring Framework
2. `application` services depend on `domain` ports only (no infrastructure)
3. `@Entity` classes reside in `domain.model` package
4. `@Service` classes reside in `application.service` package
5. `infrastructure` adapters implement domain port interfaces

This document is permanently frozen. Any future changes to student-profile-svc require a new versioned record.